

# SafeBite: Software Design Document

Architectural Blueprint, OO Design Models & UI Specification

DA-2 / REVIEW 2

September 2026

Student / Author: Arshdeep Pasricha  
Architecture: 3-Tier Containerized Web Architecture

GitHub Repo: [github.com/Arshdeep-Pasricha12/SafeBite](https://github.com/Arshdeep-Pasricha12/SafeBite)  
Academic Focus: Modularity, Cohesion, Coupling, Abstraction

## 1. Executive Summary & Problem Context

**SafeBite** is an open-access, public food-safety transparency web platform designed to bridge the severe information gap between municipal health inspection departments (such as FSSAI) and everyday dining consumers. Public diners routinely make restaurant dining decisions without visibility into routine health inspections, critical sanitation violations, or pest infestation reports buried in government archives. SafeBite centralizes municipal inspection records into an algorithmic 0–100 *Safety Score*, empowering consumers, giving certified restaurant owners a platform to showcase hygiene compliance, and enabling regulatory authorities to perform expedited audits.

## 2. Core Design Principles Applied in SafeBite

Software design principles are foundational architectural qualities that transform unstructured requirements into maintainable, scalable, and resilient systems. Rather than treating design as an academic afterthought, SafeBite's codebase reflects each principle directly:

### 2.1 Abstraction

**Definition & Application:** Abstraction hides intricate implementation details behind a declarative, contract-driven interface. In SafeBite, cryptographic security is fully abstracted inside `backend/app/core/security.py`. Higher-level authentication handlers in `api/auth.py` verify passwords and issue JSON Web Tokens (JWT) through simple functions (`verify_password()`, `create_access_token()`), remaining completely oblivious to bcrypt salt rounds, Unicode encoding schemes, or cryptographic token signing algorithms.

Similarly, on the React frontend, the centralized API gateway `frontend/src/services/api.js` wraps Axios HTTP communication behind declarative methods (e.g., `restaurantApi.getInspections(id)`), insulating UI view components from base URLs, HTTP headers, and transport protocol changes.

```
# backend/app/core/security.py – Cryptographic Abstraction in Action
def verify_password(plain_password: str, hashed_password: str) -> bool:
    try:
        # Callers receive a clean boolean; internal bcrypt encoding details are completely hidden
        return bcrypt.checkpw(plain_password.encode("utf-8"), hashed_password.encode("utf-8"))
    except Exception:
        return False

def create_access_token(data: dict, expires_delta: timedelta = None) -> str:
    to_encode = data.copy()
    expire = datetime.utcnow() + (expires_delta or timedelta(minutes=settings.ACCESS_TOKEN_EXPIRE_MINUTES))
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, settings.SECRET_KEY, algorithm=settings.ALGORITHM)
```

### 2.2 Refinement (Stepwise Decomposition)

**Definition & Application:** Refinement systematically decomposes high-level user requirements into progressively concrete, rigorously typed implementation artifacts. SafeBite follows a strict refinement pipeline from stakeholder user stories down to database storage and presentation attributes:

| Refinement Stage | Artifact Representation                                | Engineering Specifics                                               |
|------------------|--------------------------------------------------------|---------------------------------------------------------------------|
| User Story       | Story 8: "Customer views food safety score"            | Functional requirement defining customer safety risk evaluation.    |
| API Endpoint     | <code>GET /api/restaurants/{id}</code>                 | FastAPI async route returning serialized JSON payload.              |
| Pydantic Schema  | <code>RestaurantOut.safety_score: int</code>           | Runtime type validation enforcing integer values between 0 and 100. |
| ORM Entity       | <code>Restaurant.safety_score = Column(Integer)</code> | SQLAlchemy declarative model mapped to indexed relational table.    |
| Presentation     | <code>&lt;SafetyBadge score={score} /&gt;</code>       | Color-coded React component (Green: 90+, Yellow: 70-89, Red: <70).  |

**Maintainability Impact:** Refinement ensures that schema modifications are caught at compile or startup time via Pydantic type checking before corrupting the persistent data tier.

## 2. Core Design Principles Applied (Continued)

### 2.3 Modularity

**Application:** Modularity decomposes the software into autonomous, self-contained functional units that can be developed, tested, and upgraded independently. SafeBite enforces strict separation across both tiers:

**Backend Decomposition:**

- `api/`: Route endpoints (`auth.py`, `restaurants.py`, `inspections.py`, `admin.py`)
- `core/`: Cross-cutting services (`security.py`, `database.py`, `config.py`)
- `models/`: Relational ORM mapping classes (`models.py`)
- `schemas/`: Pydantic input/output contracts (`schemas.py`)

**Frontend Decomposition:**

- `pages/`: High-level screen controllers (6 distinct UI views)
- `components/`: Reusable widgets (`Navbar`, `InspectionHistory`)
- `context/`: Centralized state providers (`AuthContext`)
- `services/`: HTTP transport gateway (`api.js`)

### 2.4 High Cohesion

**Application:** Cohesion evaluates how tightly focused elements within a given module are toward a singular responsibility. SafeBite achieves *functional cohesion*—the highest recognized form in software engineering:

- `inspections.py` strictly encapsulates health inspection retrieval, violation filtering, and inspector metadata queries. It contains zero authentication logic or restaurant approval code.
- `InspectionHistory.jsx` renders the visual chronological timeline of inspections and expandable violation records. It delegates authentication checks to route wrappers and network calls to `api.js`.
- `security.py` focuses exclusively on cryptographic hash generation and token issuance. It does not interface with the database session factory or request payloads.

### 2.5 Low Coupling via Dependency Injection

**Application:** Coupling measures inter-module dependencies. High coupling creates brittle systems where altering one component breaks unrelated subsystems. SafeBite eliminates direct coupling through FastAPI's Inversion of Control (IoC) Dependency Injection system:

```
# backend/app/api/auth.py & admin.py – Low Coupling via Inversion of Control
def get_current_admin(current_user: User = Depends(get_current_user)) -> User:
    if current_user.role != "admin":
        raise HTTPException(status_code=403, detail="Operation restricted to administrators only")
    return current_user

# admin.py depends only on the contract of get_current_admin, not its concrete verification mechanics
@router.put("/restaurants/{id}/approve")
def approve_restaurant(id: str, db: Session = Depends(get_db), admin: User = Depends(get_current_admin)):
    return admin_service.approve(db, id, admin_id=admin.id)
```

**Engineering Impact:** Routers never directly instantiate database sessions or token parsers. The route handler simply declares `Depends(get_current_admin)`, and FastAPI handles dependency resolution at runtime.

### 2.6 Encapsulation & Separation of Responsibilities

**Application:** Encapsulation guarantees that internal state and mutation algorithms are sealed behind controlled accessors. On the React client, `AuthContext.jsx` encapsulates all session state. View components consume `useAuth()` to read user data or trigger authentication mutations, but never access browser `localStorage` directly.

On the data tier, Pydantic schemas enforce a 3-stage boundary: `UserBase` (shared attributes), `UserCreate` (contains raw incoming plaintext password; never returned), and `UserOut` (serializes response payload with database ID while stripping sensitive cryptographic hashes).

## Summary of Principles and Impact on Maintainability

| Design Principle | SafeBite Architectural Implementation                                                           | Measurable Impact on System Maintainability                                          |
|------------------|-------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Abstraction      | Centralized <code>security.py</code> and frontend <code>services/api.js</code>                  | Allows replacing bcrypt/JWT or switching Axios to fetch without modifying UI pages.  |
| Refinement       | User Story → Pydantic Schema → SQLAlchemy Model → UI Component                                  | Prevents database schema drift and establishes unambiguous end-to-end type safety.   |
| Modularity       | Domain-partitioned routes ( <code>auth</code> , <code>restaurants</code> , <code>admin</code> ) | Enables parallel team development and localized bug isolation.                       |
| Cohesion         | Single-responsibility files (e.g., <code>InspectionHistory.jsx</code> )                         | A feature modification to violation rendering touches exactly one presentation file. |
| Low Coupling     | FastAPI <code>Depends()</code> injection and unified Axios gateway                              | Enables mock testing without live databases and decouples auth rules from routes.    |
| Encapsulation    | <code>AuthContext</code> token shielding and Pvdantic DTO                                       | Guarantees zero accidental leakage of credentials or                                 |

3. High-Level Architecture Specification

SafeBite employs a **3-Tier Layered Architecture** with containerized deployment orchestration using Docker Compose. The architecture enforces unidirectional downward dependency flow: the Presentation Layer interacts exclusively with the Business Logic Layer via structured JSON REST APIs, while the Business Logic Layer interfaces with the Data Layer through an Object-Relational Mapping (ORM) abstraction.

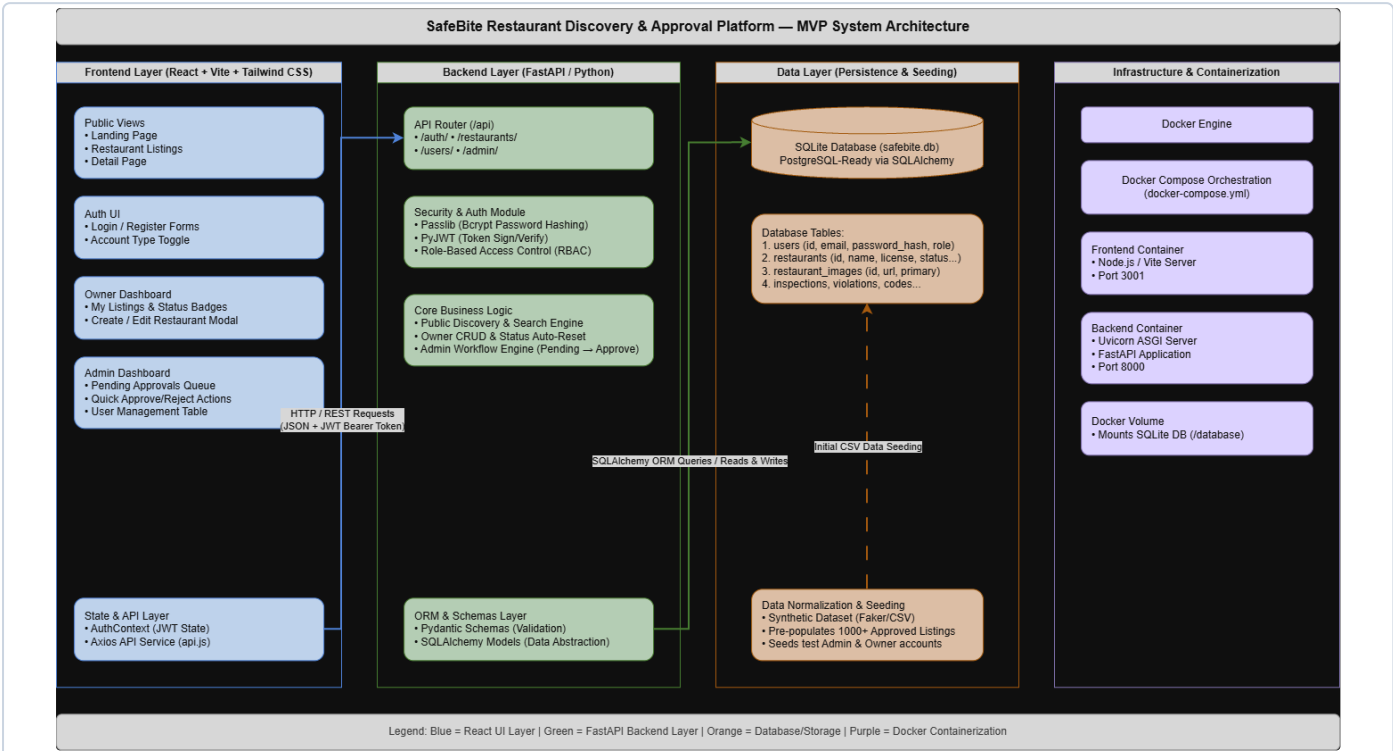


Figure 1: SafeBite 3-Tier Layered Architecture and Containerized Runtime Orchestration (Source: design/architecture/SafeBite\_Architecture.drawio.png)

Why This Architecture Was Chosen

- 1. Independent Lifecycle & Deployment:** The React 18 frontend and FastAPI backend run in dedicated Docker containers connected via a bridge network. UI updates can be deployed without restarting the REST API.
- 2. Asynchronous Performance:** FastAPI with Uvicorn handles concurrent I/O requests asynchronously, essential for serving complex multi-table inspection queries under peak consumer traffic.
- 3. Testability & Component Isolation:** Business rules (safety score weighting, RBAC verification) reside strictly in the API layer, allowing automated testing without launching a browser.

Layer Responsibilities

- Presentation Layer (Port 3001):** React 18, Vite, Tailwind CSS, Lucide icons. Responsible for view rendering, user input validation, and consuming REST payloads.
- Business Logic Layer (Port 8000):** FastAPI, Python 3.12, PyJWT, bcrypt. Enforces authorization, schema validation, score calculation algorithms, and business workflows.
- Data Persistence Layer:** SQLite (development) / PostgreSQL (production) managed via SQLAlchemy 2.0 ORM with relational integrity constraints.

Traceability Matrix: Requirements to Architecture Mapping

| Requirement ID     | User Story / Functional Need                    | Architectural Subsystem & Modules                | Layer & Technology                             |
|--------------------|-------------------------------------------------|--------------------------------------------------|------------------------------------------------|
| Stories 1, 2, 3    | User Registration, Login, and Role-Based Access | api/auth.py , core/security.py , AuthContext.jsx | Business Logic & Presentation (JWT / Bcrypt)   |
| Stories 4, 5, 6    | Browse, Filter, and Search Restaurants          | api/restaurants.py , RestaurantListing.jsx       | Cross-Tier (FastAPI Async + React Grid)        |
| Stories 8, 9, 10   | Safety Score Display & Inspection Timeline      | api/inspections.py , InspectionHistory.jsx       | Data & Presentation (Relational Violations)    |
| Stories 13, 14, 15 | Owner Restaurant Registration & Status Tracking | api/restaurants.py , OwnerDashboard.jsx          | Business Tier (Draft to Pending state machine) |
| Stories 18, 19, 20 | Administrator Verification & Approval Cockpit   | api/admin.py , AdminDashboard.jsx                | Business Tier (RBAC Guarded Operations)        |
| Story 24           | Government Inspection CSV Data Pipeline         | create_database_from_csv.py                      | Data Layer (Batch Ingestion & Scoring Engine)  |

## 4. Detailed & Object-Oriented Design (UML Models - Part 1)

In conformance with the Software Engineering syllabus, SafeBite's structural and behavioral properties are modeled through formal Unified Modeling Language (UML) diagrams. Each diagram captures a distinct viewpoint of the system's operational architecture.

### 4.1 Component Diagram (Subsystem Packaging)

**Academic Justification:** The Component Diagram models the physical and logical modularity of SafeBite. It illustrates how the React SPA client, FastAPI application routers, security subsystem, and relational database collaborate across well-defined REST interfaces.

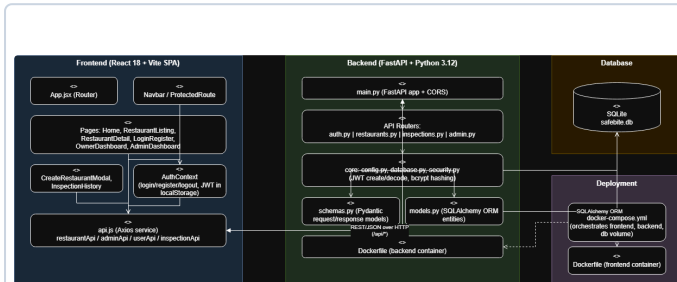


Figure 2: Component Architecture & Interface Connectors (Source: [design/architecture/component.drawio.png](#))

**Key Insight:** The frontend is completely decoupled from the data layer. Components communicate strictly through the `/api` JSON gateway.

## 4.2 Sequence Diagram (Authentication & Access Control)

**Academic Justification:** Sequence diagrams capture time-ordered object interactions. Authentication is SafeBite's most intricate multi-step protocol, spanning client storage, cryptographic password verification, JWT generation, and role-based route gating.

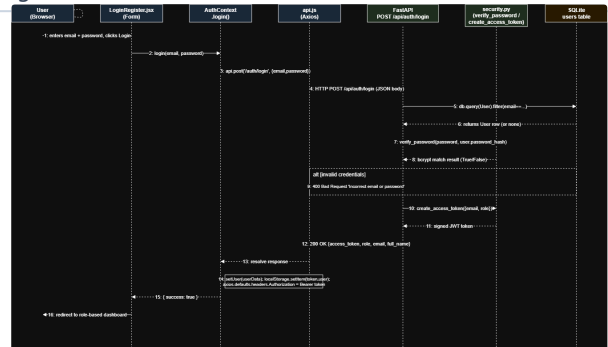


Figure 3: Sequence Flow for Login & RBAC Request Execution (Source: [design/architecture/sequence.drawio.png](#))

**Execution Path:** Credential POST → Bcrypt Hash Check → JWT Issuance → LocalStorage Persist → Bearer Authorization Header on subsequent queries.

### 4.3 Key Object-Oriented Design Patterns Applied

## 1. Repository / ORM Pattern

**Implementation:** Encapsulated via SQLAlchemy `Session` and model classes.

**Benefit:** Abstracts raw SQL queries into object interactions (e.g., `db.query(Restaurant).filter(...)`). Prevents SQL injection and allows swapping database dialects with zero model rewrites.

## 2. Data Transfer Object (DTO)

**Implementation:** Pydantic models (`BaseModel`) partitioned into `Create`, `Update`, and `Out` contracts.

**Benefit:** Decouples internal database schemas from external API contracts. Sensitive attributes (like `password_hash`) are structurally barred from serializing into client responses.

### 3. Context & Guard Patterns

**Implementation:** React `AuthContext` and `ProtectedRoute.jsx`.

**Benefit:** Eliminates "prop drilling" by broadcasting user session state globally. Route guards provide declarative access control, automatically intercepting unauthenticated or unauthorized role access.

#### 4.4 Use Case Modeling (Actor Interaction Matrix)

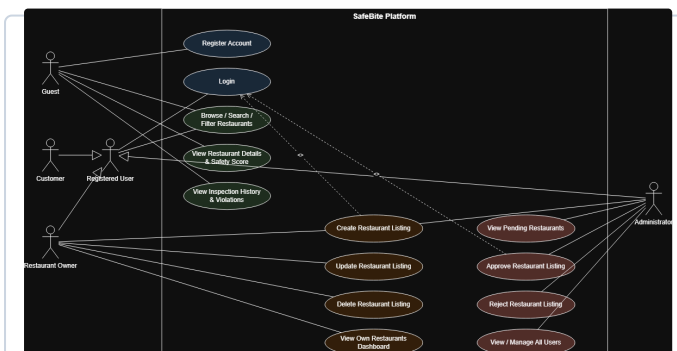


Figure 4: SafeBite Actor Use Case Diagram (Source: [design/architecture/usecase.drawio.png](#))

The Use Case model delineates four primary system actors with distinct privilege domains:

- **Customer:** Unauthenticated discovery of restaurants, safety score filtering, detailed violation inspection history.
- **Restaurant Owner:** Authenticated creation and modification of establishment profiles; real-time tracking of verification statuses.
- **Administrator:** Global audit control, pending application approval/rejection with mandatory feedback notes, and user account lifecycle control.
- **Health Inspector:** Publishing routine and follow-up inspection reports, logging violation codes, and assigning compliance ratings.

## 4. Detailed & Object-Oriented Design (UML Models - Part 2)

#### 4.5 Class Diagram (Domain Model & Hierarchy)

**Academic Justification:** The Class Diagram models static structural relationships. It documents domain entities (`User`, `Restaurant`, `Inspection`, `Violation`, `Inspector`), Pydantic inheritance graphs, and router controller signatures.

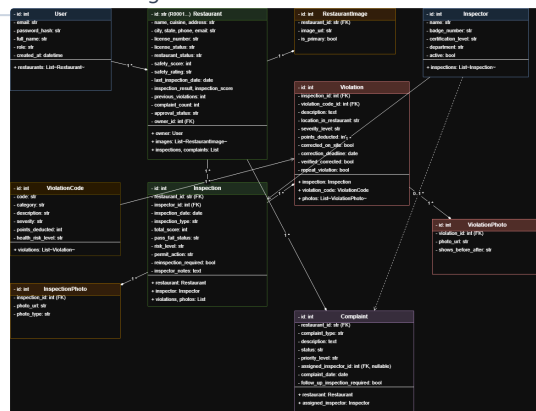


Figure 5: Domain Entity Classes & Inheritance Hierarchy (Source: design/architecture/Class Diagram.drawio.png)

**Key Relationship:** **Restaurant** maintains a 1-to-many relationship with **Inspection**, which cascades to multiple **Violation** entries linked to standardized **ViolationCode** entities.

#### 4.7 State Chart (Restaurant Application Lifecycle)

**Academic Justification:** Models the discrete operational states of a restaurant listing. Transition guards prevent unapproved establishments from being exposed to public search queries.

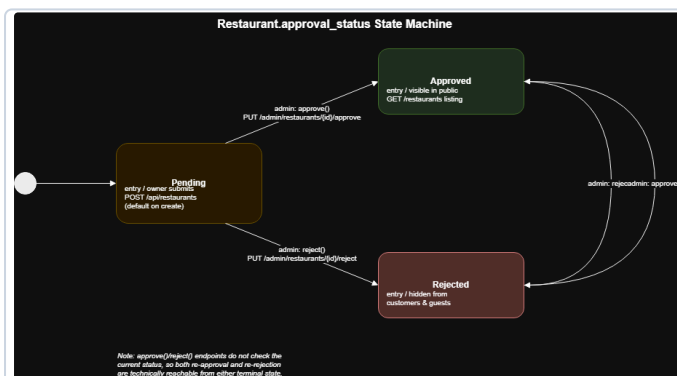


Figure 7: State Chart of Restaurant Verification (Source: [design/architecture/statechart.drawio.png](#))

**State Transitions:** Draft → Pending Review → Approved (Active) OR Rejected (Remediation Required) → Resubmitted.

#### 4.6 Entity-Relationship (ER) Diagram

**Academic Justification:** The ER diagram defines persistent database schemas, primary/foreign key relationships, and data normalization across 10 relational entities, guaranteeing referential integrity.

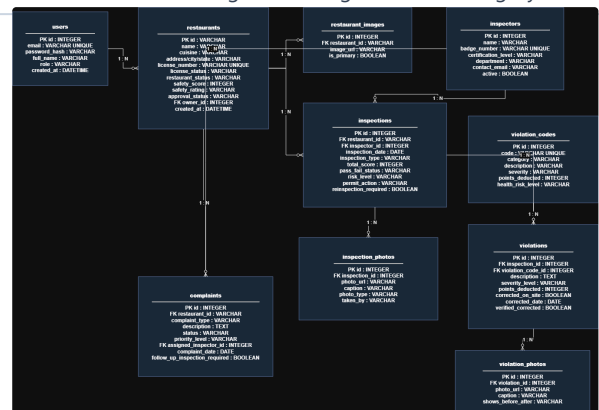


Figure 6: Relational Database Schema & Constraints (Source: [design/architecture/ER.drawio.png](#))

**Referential Integrity:** Deleting an inspection automatically cascades deletions to child violation records ( `cascade="all, delete-orphan"` ), preventing orphan data.

#### 4.8 Activity Diagram (Inspection & Scoring Engine)

**Academic Justification:** Documents control flow for ingestion, rule validation, penalty point calculation, and algorithmic score normalization (0–100 scale).

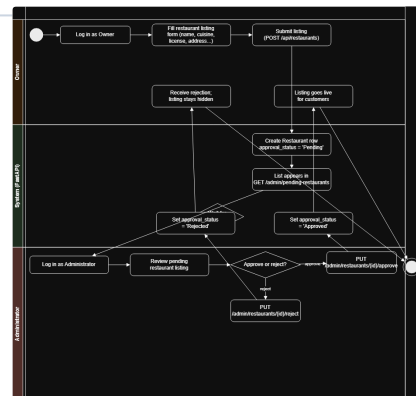


Figure 8: Activity Diagram of Inspection Scoring Engine (Source: design/architecture/activity.drawio.png)

**Algorithm:** Raw points start at 100. Critical violations subtract 15–20 pts; major subtract 8–10 pts; minor subtract 2–4 pts. Normalized score determines letter grade (A/B/C/D).

## 5. User Interface (UI/UX) Design (Part 1: Consumer Interfaces)

SafeBite's frontend interface is designed around four core usability principles: **Simplicity** (uncluttered visual hierarchy), **Consistency** (standardized Tailwind typography, color tokens, and button hierarchies), **Error Prevention** (form constraints and real-time inline validation), and **Instant Feedback** (color-coded badges, skeleton loaders, and responsive toasts).

Screen 1: Authentication Portal

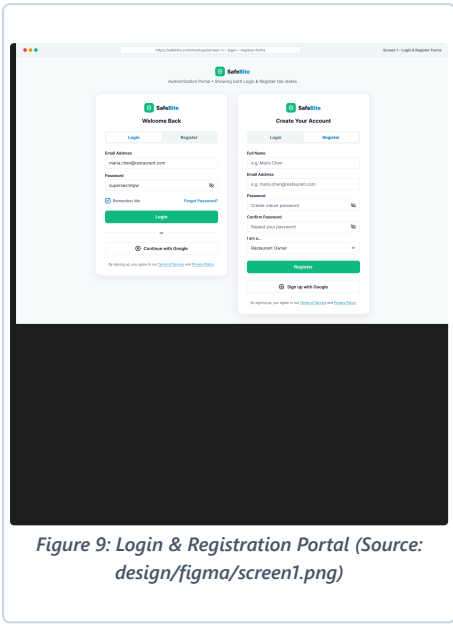


Figure 9: Login & Registration Portal (Source: design/figma/screen1.png)

Screen 2: Home / Landing Page

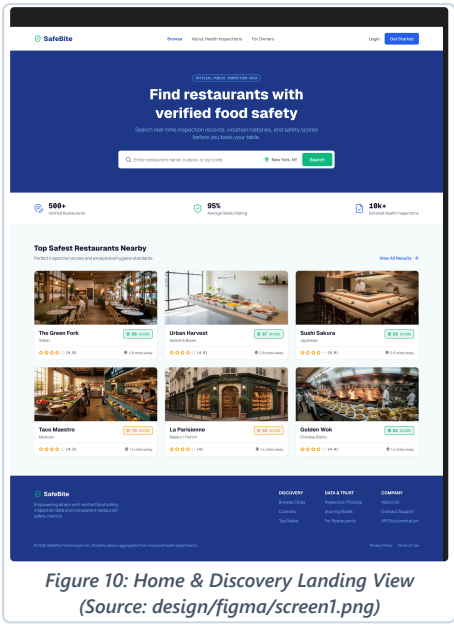


Figure 10: Home & Discovery Landing View (Source: design/figma/screen1.png)

Screen 3: Restaurant Listing & Filter

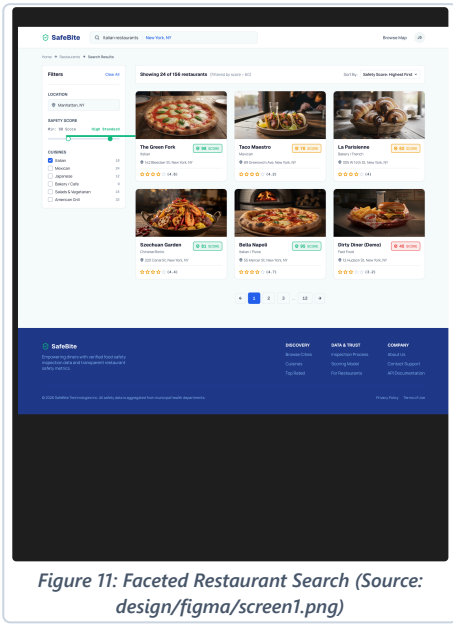


Figure 11: Faceted Restaurant Search (Source: design/figma/screen1.png)

**Purpose:** Unified gateway for customer, restaurant owner, and auditor authentication.

**User Stories:** 1 (Register), 2 (Login), 3 (RBAC Access).

**Key UX Decisions:**

- Tabbed switcher between Login and Register eliminates jarring page reloads.
- Role dropdown with descriptive text prevents accidental role misassignment.
- Password visibility toggle minimizes credential input errors.

**Flow:** Select Role → Enter Credentials → JWT Received → Role-Specific Redirect.

**Purpose:** Public landing view introducing the safety platform and quick search.

**User Stories:** 22 (Public Platform Navigation & Search).

**Key UX Decisions:**

- Hero search bar is front-and-center, accepting restaurant names, locations, and cuisines.
- Social trust metrics (500+ Verified Restaurants, 95% Data Accuracy) establish credibility.
- Featured cards showcase top-scoring eateries with instant green score badges.

**Flow:** Search input or category click → Instant redirection to filtered catalog.

**Purpose:** Catalog browsing with interactive safety and geographic filtering.

**User Stories:** 4 (Browse), 5 (Faceted Filter), 6 (Search).

**Key UX Decisions:**

- Sticky left sidebar allows multi-facet filtering (Safety Score slider, Cuisines, Cities).
- Active filter tags provide clear visibility into applied parameters with 1-click removal.
- Color-coded safety badges (Green: 90+, Yellow: 70-89, Red: <70) offer instant risk clarity.

**Flow:** Adjust filter slider → Debounced API query → Paginated card updates.

### Summary of UI Usability Engineering

The consumer interface addresses cognitive load by organizing dense sanitary data into standardized visual metaphors. Rather than presenting raw municipal inspection transcripts, SafeBite's listing views present high-level synthesized scores with clear color semantics, reserving granular violation details for dedicated secondary pages (progressive disclosure).

## 5. User Interface (UI/UX) Design (Part 2: Operational & Admin Interfaces)

Operational screens cater to specialized personas (Restaurant Owners and Public Health Administrators), requiring high-density informational layouts, real-time status indicators, and streamlined approval workflows.

### Screen 4: Restaurant Detail & History

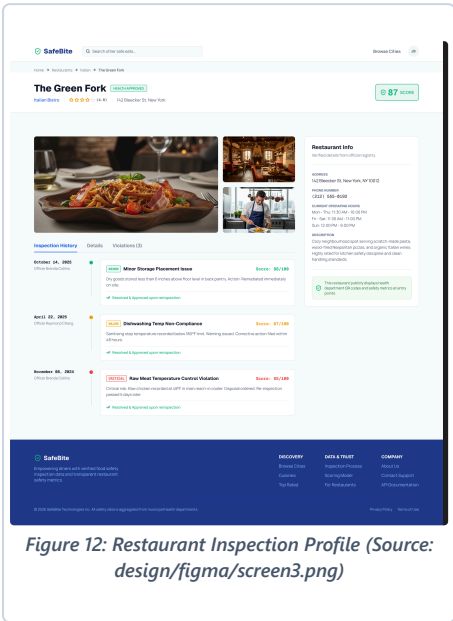


Figure 12: Restaurant Inspection Profile (Source: design/figma/screen3.png)

**Purpose:** In-depth safety dossier displaying official inspection history.

**User Stories:** 7 (Profile), 8 (Score), 9 (Timeline), 10 (Violations).

**Key UX Decisions:**

- Prominent circular 87/100 score gauge with official health department certification badge.
- Chronological timeline showing inspection dates, inspector names, and outcome grades.
- Expandable violation cards labeled with severity tags (Critical, Major, Minor).

**Flow:** Click listing card → Load full profile → Expand violation for remediation notes.

### Screen 5: Owner Management Dashboard

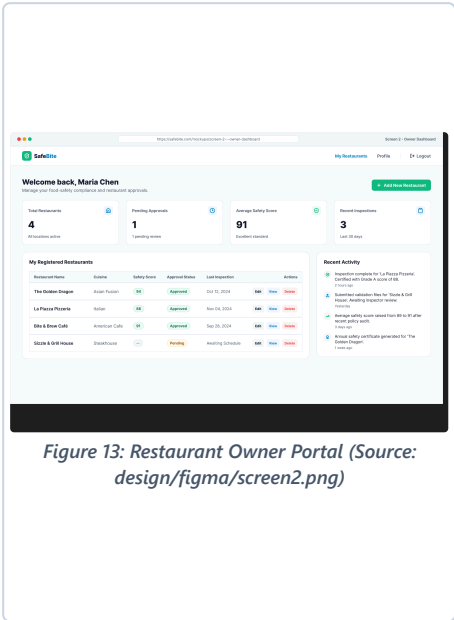


Figure 13: Restaurant Owner Portal (Source: design/figma/screen2.png)

**Purpose:** Establishment management console for restaurant proprietors.

**User Stories:** 13 (Register Listing), 14 (Edit Details), 15 (Status).

**Key UX Decisions:**

- Metric cards show total properties, pending reviews, and average compliance score.
- Tabular listing with explicit status badges: *Approved* (green), *Pending* (amber), *Rejected* (red).
- Modal form for "Add New Restaurant" prevents navigating away from active dashboard context.

**Flow:** Login as Owner → View listing status → Submit new location for audit review.

### Screen 6: Admin System Cockpit

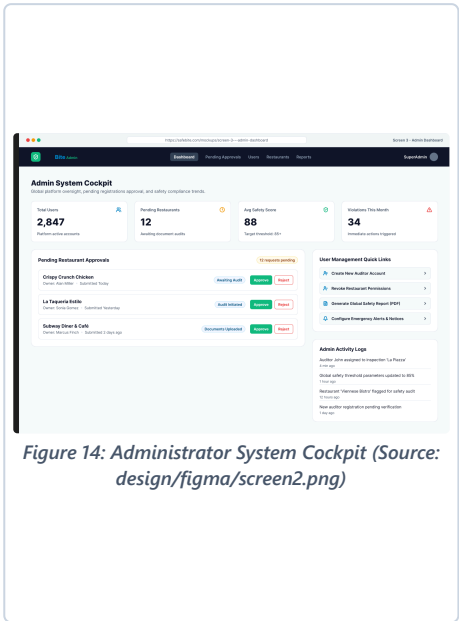


Figure 14: Administrator System Cockpit (Source: design/figma/screen2.png)

**Purpose:** Centralized supervisory cockpit for regulatory verification.

**User Stories:** 17 (Review), 18 (Approve), 19 (Reject), 20 (Users).

**Key UX Decisions:**

- Operational telemetry cards: Total Users, Pending Approvals, Violations logged.
- Side-by-side *Approve* and *Reject* action triggers minimize workflow friction.
- Rejection modal enforces entering mandatory corrective action instructions for the owner.

**Flow:** Review pending submission → Inspect documentation → 1-Click Approve.

### Error Prevention and Feedback Mechanisms Across Dashboards

Administrative actions carry system-wide consequences. Deleting records or rejecting applications triggers confirmation dialogs with irreversible action warnings. Successful administrative approvals trigger real-time notification toasts and update the global audit log, ensuring complete transparency and operational accountability across all user tiers.

## 6. Architectural Decisions, Trade-Offs & Code Artifacts

Every architectural choice represents an engineered compromise between competing quality attributes. The following five foundational decisions guided the construction of SafeBite:

| Summary of 5 Key Architectural Decisions |                       |                              |                                                                                                          |
|------------------------------------------|-----------------------|------------------------------|----------------------------------------------------------------------------------------------------------|
| Decision                                 | Chosen Solution       | Rejected Alternative         | Engineering Justification & Trade-Off                                                                    |
| 1. API Framework                         | FastAPI (Python 3.12) | Flask / Django REST          | Native Pydantic validation, automatic OpenAPI/Swagger docs, async request throughput.                    |
| 2. Data Persistence                      | SQLAlchemy 2.0 ORM    | Raw SQLite / Raw SQL         | Strict type safety, prevention of SQL injection, effortless dialect portability (SQLite to Postgres).    |
| 3. Client State                          | React Context API     | Redux Toolkit                | Global state is strictly confined to user session and JWT. Context avoids massive Redux boilerplate.     |
| 4. Authentication                        | Stateless JWT Tokens  | Server-side Sessions (Redis) | Horizontal scalability without shared session memory; standard Bearer headers enable mobile expansion.   |
| 5. Deployment                            | Docker Compose Stack  | Bare-metal Localhost         | Deterministic multi-container isolation; guarantees identical runtime across all host operating systems. |

## 7. Production Code Snippets Demonstrating Architectural Principles

### Snippet A: Pydantic DTO Contract Separation (schemas.py)

```
# Separation of Concerns: Base vs Input vs Output Contracts
class UserBase(BaseModel):
    email: EmailStr
    full_name: str
    role: str = Field(..., description="customer, owner, admin")

class UserCreate(UserBase):
    password: str # Accepted only during registration

class UserOut(UserBase):
    id: int
    created_at: datetime
    # Crucial: password and password_hash are physically excluded
    class Config:
        from_attributes = True
```

**Principle:** *Encapsulation & Security.* Input passwords never leak into outgoing client serializations.

### Snippet B: RBAC Dependency Injection (api/auth.py)

```
# Low Coupling: Inversion of Control with FastAPI Depends()
def get_current_admin(current_user: User = Depends(get_current_user)):
    if current_user.role != "admin":
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="Operation restricted to administrators"
        )
    return current_user

# Usage: Single parameter declares complete RBAC guard
@router.put("/restaurants/{id}/approve")
def approve(id: str, admin: User = Depends(get_current_admin)):
    ...
```

**Principle:** *Low Coupling & High Cohesion.* Admin authorization rules are centralized in a callable guard.

### Snippet C: SQLAlchemy Domain Entity (models/models.py)

```
# Domain Model Cohesion & Relational Cascading
class Inspection(Base):
    __tablename__ = "inspections"
    id = Column(Integer, primary_key=True, index=True)
    restaurant_id = Column(String, ForeignKey("restaurants.id"))
    total_score = Column(Integer, nullable=False)
    risk_level = Column(String, nullable=False)

    # Cohesive relationships with automated cascade cleanup
    restaurant = relationship("Restaurant", back_populates="inspections")
    violations = relationship("Violation", back_populates="inspection",
                             cascade="all, delete-orphan")
```

**Principle:** *High Cohesion.* Captures all inspection attributes and relational dependencies in one definition.

### Snippet D: Centralized HTTP Service Layer (services/api.js)

```
// Client Abstraction Layer: Decouples UI from Axios
import axios from 'axios'
export const api = axios.create({ baseURL: '/api' })

export const restaurantApi = {
  getAll: (params) => api.get('/restaurants', { params }),
  getById: (id) => api.get(`/restaurants/${id}`),
  getInspections: (id) => api.get(`/restaurants/${id}/inspections`)
}

export const adminApi = {
  approve: (id) => api.put(`/admin/restaurants/${id}/approve`)
}
```

**Principle:** *Abstraction.* Components call methods; they never manage transport settings directly.

## 8. Maintainability, Extensibility & Future Evolution

A premier hallmark of professional software architecture is minimizing the cost and risk of future modifications. SafeBite adheres to the **Open/Closed Principle** (open for extension, closed for modification):

### Scenario 1: Adding Notifications

**Objective:** Send email notifications upon restaurant approval.

**Steps:**

- Create `api/notifications.py` with dispatch routes.
- Add `Notification` entity to `models/models.py`.
- Mount router in `main.py`: `app.include_router(...)`.
- Add `notificationApi` to `services/api.js`.

**Impact:** Zero modifications required to existing auth or restaurant handlers.

### Scenario 2: Database Migration

**Objective:** Upgrade from local SQLite to enterprise PostgreSQL.

**Steps:**

- Update `DATABASE_URL` in `core/config.py`: `postgresql://user:pass@db:5432/safebite`
- Install `psycopg2-binary` driver in `requirements.txt`.
- Launch PostgreSQL service in `docker-compose.yml`.

**Impact:** All models, queries, and routers remain 100% untouched.

### Scenario 3: Module Portability

**Objective:** Reuse the authentication module in an external service.

**Steps:**

- Copy `core/security.py` into the new Python project.
- Copy `components/ProtectedRoute.jsx` into new React apps.

**Impact:** `security.py` has zero internal dependencies; it depends only on `bcrypt` and `PyJWT`, making it instantly pluggable.

## 9. Software Quality Engineering & Testing Architecture

### 9.1 Architecture Support for Testability

FastAPI's dependency injection architecture allows swapping production services with mocks during unit and integration test execution:

```
# backend/test_api.py - Dependency Override Pattern
app.dependency_overrides[get_db] = override_test_db
# API tests run against an isolated in-memory SQLite database
response = client.get("/api/restaurants")
assert response.status_code == 200
```

- **Pure Functions:** Cryptographic functions in `security.py` are pure functions without I/O side effects, enabling millisecond unit testing.
- **Mockable Frontend:** Decoupling React pages behind `services/api.js` allows testing view components against mock JSON fixtures without backend execution.

| 9.2 Evaluation Against ISO/IEC 9126 Quality Attributes |                                                                                               |
|--------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| Quality Factor                                         | Architectural Support & Design Implementation                                                 |
| Maintainability                                        | Functional cohesion, modular packaging, and centralized API gateways prevent ripple bugs.     |
| Reliability                                            | Pydantic schemas reject malformed inputs before reaching SQL tables; ORM stops SQL injection. |
| Usability                                              | Tailwind design tokens, consistent color-coded safety metrics (0-100), and responsive layout. |
| Portability                                            | Docker multi-container orchestration and SQLAlchemy database abstraction.                     |
| Efficiency                                             | FastAPI asynchronous event loop and indexed relational database queries on key fields.        |

### Appendix: Digital Assignment 2 Submission & Asset Directory

- **GitHub Repository:** <https://github.com/Arshdeep-Pasricha12/SafeBite> (All design artifacts and source committed)
- **Design Directory ( /design/ ):** Contains 7 editable `.drawio` source diagrams, 8 PNG diagram exports in `/design/architecture/`, and 6 Figma screens in `/design/figma/`.
- **README Documentation:** Section 5 "Software Design" updated with diagrams and architectural summary table.